

Vehicle Detection and Counting Using YOLO

Abstract

A traffic video already contains some information about the road, yet reading that information manually takes some time. This chapter describes a computer vision pipeline implemented in the You Only Look Once (YOLO) framework. It detects vehicles in every frame, assigns a class and confidence value to it, tracks detections over time, and logs a count when the tracked vehicle crosses a virtual line or region. This section talks about the design of the pipeline, data preparation, inference, tracking, implementation, and evaluation. The prototype is created for educational and research purposes. Its behavior will change depending on the camera position, illumination, traffic load, occlusion, model parameters, and video quality. The project report defines the pipeline evaluation procedure, however, does not include the completed accuracy and frames-per-second benchmark. Therefore, this chapter describes the pipeline workflow without adding a performance figure that cannot be confirmed.

Keywords: *YOLO, vehicle detection, object detection, computer vision, vehicle counting, object tracking, traffic monitoring, deep learning.*

1. Introduction

1.1 Background

It is easy to observe a busy road for several minutes. It is more difficult to measure the traffic consistently over a long period of time. Traffic authorities require vehicle counts and categories for congestion studies, road planning, and safety analyses. The manual observation is applicable for a brief survey, but becomes tedious when the recording is long or there are multiple cameras. There are fixed sensors and inductive loops as an alternative, but they require installation in specific points of the road. This project assumes that the traffic camera serves as a source of evidence and uses YOLO, tracking, and line-crossing logic to convert that evidence into a count.

The camera provides only frames. They do not contain explanations about what they represent, and the interpretation of that information may change when vehicles overlap, get smaller in the distance, or move with different speeds. The processing stage first sends a frame to YOLO, filters out unnecessary detections, and sends boxes of the rest of the objects to the tracker. The counting takes place when the movement of an identified vehicle satisfies the selected line or region rule.

1.2 Vehicle Detection and Counting

In one frame, vehicle detection performs two tasks: identifies the vehicle and estimates its location. The resulting information is saved as a bounding box, class label, and confidence value. The available classes depend on the selected model or custom training configuration; the project assumes common categories such as cars, motorcycles, buses, and trucks. These detections will be used later to analyze vehicle movements through the scene.

A video cannot be counted by combining boxes. One car might appear in multiple consecutive frames, thus increasing the total count excessively. The tracker matches those detections and uses one identifier for the active vehicle. Only the trajectory that crosses the configured line or region triggers

the counting event. Detection, tracking, and counting are therefore separate steps of the same workflow.

1.3 Motivation and Contribution

The practical motivation for the project was very straightforward: can the ordinary traffic footage provide a count verifiable by another person? A single detector could not answer that question. The implementation of the project combines video acquisition, YOLO inference, confidence filtering, identity association, line-crossing logic, frame annotation, and numerical summarizing. If the count changes, the user can verify the boxes, labels, identifiers, and boundary instead of taking into account only the number itself.

The main contribution of the project is the connection between these components. YOLO provides observations made in individual frames. Tracking adds a small history to those observations, and the boundary logic converts the trajectory into a counting event. This chapter also considers detection quality and counting quality as separate factors. A detector can identify a vehicle while the tracker loses its identity. The tracker also can work correctly while the incorrectly placed line generates a wrong event. Separating those cases simplifies testing the prototype.

2. Literature Review

2.1 Traditional Vehicle Detection Methods

The older traffic monitoring systems often applied background subtraction, frame differencing, optical flow, edge detection, or contour analysis. Background subtraction distinguishes the moving regions from the relatively stationary background. Frame differencing compares successive images, and optical flow estimates the apparent motion of the pixels. Such techniques are good building blocks, but they are very dependent on the camera and scene. A shadow, change of the illumination, or camera motion can alter the result even if the traffic does not change.

The threshold that works for one road view might need to be changed for another one. The dense traffic causes another issue since the vehicles may overlap in the image. Visual representations learned by the algorithms reduce the dependence on the scene-specific assumptions. YOLO is helpful in this case since localization and classification are done in one inference of the network [1] to [6].

2.2 Deep Learning-Based Object Detection

The convolutional neural network learns visual representations in stages. The early layers react to the simple structures such as edges and textures. The latter layers combine these reactions into features that describe the object. The object detector uses these features for two tasks: classifying the object and estimating its location. The two-stage detector divides the region proposal from the classification, and single-stage detector produces both of these results in one prediction step. This difference is important because the model has to perform another inference for each frame of the video.

YOLO proposed the unified approach to direct prediction of the object locations and classes from an image [1]. The later versions modified the architecture and training procedure [2] to [6]. This project does not introduce another YOLO model. It focuses on the less obvious part of the application: maintaining the object identity after the detector has worked and converting this identity into the count that can be checked against the video.

2.3 Object Tracking and Vehicle Counting

The multi-object tracker receives the detections made in successive frames and matches them by the identity. The location, motion, and overlap help in making this association. The SORT and Deep SORT are light-weight examples, while ByteTrack and similar methods also use low-confidence observations for track formation [7] to [9]. This step is important since the final count depends on identity continuity, and not just on the number of boxes returned by YOLO.

The counting logic is based on the geometric rule. A line or region is defined in a proper point of the road scene, and the center of each tracked box is checked as the video progresses. The movement from one side of the boundary to another is logged as the crossing. The identifier is saved to ensure that the same movement is not counted again. The same logic can be used to separate totals by the vehicle class or direction, if these fields are available.

2.4 Research Gap and Motivation for the Present Work

Vehicle localization is only the first step in the counting problem. The missed box, identity change, partial occlusion, incorrect line placement, or poor camera angle can alter the final count.

Therefore, the user needs to verify the detections, tracks, and crossing events independently. This project combines those verification procedures in one pipeline for traffic video or a suitable camera stream. It supports common vehicle categories and keeps the intermediate results visible. While the video is playing, the user can ask about the vehicles that have been detected, about the stability of their identifiers, and the reasons of count changing. Since the detector, tracker, and counting rule are separate, each of them can be tuned or replaced later.

3. Problem Formulation

3.1 Problem Statement and Objectives

Multiple vehicles may occupy the same frame and move in different directions. The size of their representation alters depending on the distance from the camera, and one vehicle can partially occlude another one. The video needs to be decoded, frames need to be processed within available resources, and weak detections need to be filtered out to be able to match observations. Then the counting logic has to decide whether the trajectory has crossed the boundary and ignore the second event of the same vehicle.

The problem solved by this project is the design of the YOLO-based solution that accepts traffic video, detects and classifies vehicles, tracks them across consecutive frames, and counts them when they cross a predefined virtual line or region. The output has to be enough informative to allow the user to check the boxes, identifiers, and totals.

- To detect vehicles automatically from traffic images or video.
- To classify the detected vehicles into supported categories.
- To use YOLO for efficient frame-level object detection.
- To maintain unique identifiers for vehicles across consecutive frames.
- To implement line or region-based vehicle counting.
- To prevent the repetition of counting the same tracked vehicle.
- To calculate total, class-wise, and directional vehicle counts where supported.

- To display the detection and counting results in a visible form.

3.2 Scope, Assumptions, and Requirements

The scope is limited to vehicle detection, classification, tracking, and counting in recorded traffic video or a suitable camera stream. It does not try to manage the whole road network. The camera is assumed to be relatively stationary, vehicles have to be sufficiently visible for the selected model, and the scene has to provide a sensible location for the counting line or region. The change in viewpoint or visibility affects all subsequent stages, therefore, these assumptions are a part of the experimental record.

ID	Functional requirement	Acceptance criteria
F1	Accept traffic video	Video is loaded successfully
F2	Detect vehicles	Boxes and labels are displayed
F3	Track vehicles	Identifiers are unique
F4	Define counting region	Line or region is visible
F5	Count vehicles	Count is updated after a valid crossing
F6	Classify vehicles	Supported vehicle classes are displayed
F7	Display results	Annotated video and statistics are generated
F8	Export results	Count data or report is saved

Table 1: Functional requirements for the vehicle detection and counting system

Quality attribute	Requirement
Correctness	Reject incompatible inputs, preserve geometry, and keep reference labels outside inference.
Reproducibility	Record the dataset and model revision, device, input shape, and model name.
Usability	Display the visible status, labeled controls, keyboard focus, and readable states.
Performance	Use bounded processing and record the selected input and execution settings.
Privacy	Keep uploaded studies on the local computer and use local processing if possible.
Safety	Display the output as traffic analysis and avoid unsupported operational claims.

Table 2: Quality and operational requirements

4. Materials and Methods

A frame goes through a series of processing operations: video acquisition, preparation, YOLO inference, confidence filtering, tracking, line-crossing analysis, and presentation. Each step passes different information to the next. The next sections describe these stages and how the project can be evaluated.

4.1 System Architecture

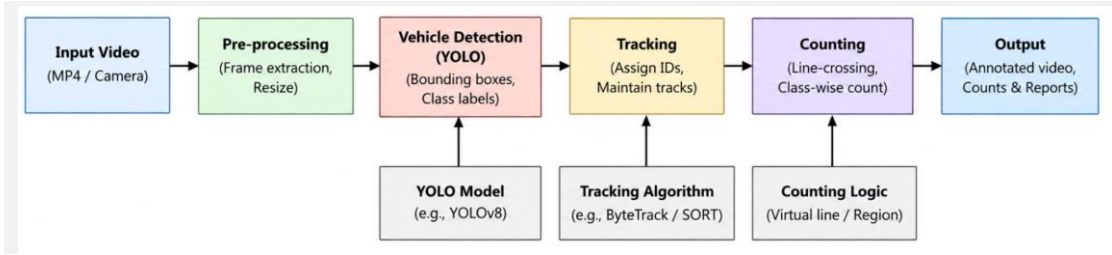


Fig 4.1 End-to-end architecture of the vehicle detection and counting system using YOLO.

Source: Original project report.

The pipeline includes four processing modules. The input module acquires frames from a video or camera stream. YOLO analyzes each frame and provides boxes, labels, and confidence values of the vehicle detections. The tracker matches the accepted boxes with active trajectories and assigns identifiers to them. Finally, the counting module checks those trajectories for the virtual line or region. The last stage displays the result and statistics.

This separation is helpful when the output is wrong. The missing box indicates the detection problem. The change of identifier indicates the tracking problem. The wrong update might come either from the crossing rule or from the line position. In short, the detector describes the frame, the tracker adds the time component, and the counter analyzes the movement.

Stage	Primary responsibility	Failure to inspect
Input	Video acquisition and frame extraction	Incompatible or corrupt video
Detection	Vehicle localization and classification	Missed or false detections
Tracking	Maintain vehicle identities	Identity changes or lost tracks
Counting	Detect line or region crossings	Duplicate or missed counts
Presentation	Display evidence and statistics	Invisible or unverifiable output

Table 3: Pipeline stages and failure modes

4.2 Dataset and Traffic Video

The input may be a public traffic dataset or a custom road surveillance video. The selected videos have to include the vehicle categories and scenes conditions relevant to the future use of the project. A pretrained YOLO model may already cover common classes, allowing the project to concentrate on detection, tracking, and counting. If the target road differs considerably from the model data, additional training or fine-tuning may be required.

For each experiment, the project has to log the video source, camera viewpoint, frame dimensions, processing rate, model revision, confidence threshold, tracker settings, and coordinates of the counting line. This information allows to define if the change in the count was caused by the code, model, scene, or geometry.

Parameter	Description	Details
Dataset Type	Traffic video dataset	Real-world traffic videos recorded from roadside cameras.
Input Format	Video files	MP4 / AVI, 1920 × 1080 (varies by source).
Video Duration	Per video	1–10 minutes (varies by location).
Frame Rate	FPS	25 or 30 FPS.
Vehicle Classes	Target objects	Car, Motorcycle, Bus, Truck.
Total Videos	Dataset size	10 traffic videos (5 for training, 3 for validation, 2 for testing).
Annotation	Bounding boxes	Manually annotated using Labellmg / Roboflow.
Detection Model	Object detection	YOLO (e.g., YOLOv8).
Tracking Method	Multi-object tracking	ByteTrack / SORT.
Counting Method	Vehicle counting	Virtual line crossing.
Output	System results	Annotated video, class-wise counts and reports.

Table 2. Dataset and traffic-video configuration for vehicle detection and counting.
Source: Authors' implementation.

Fig 4.2 Dataset and traffic-video configuration for vehicle detection and counting.
Source: Original project report.

4.3 Data Preparation Protocol

Preparation starts from opening the video and verifying that it can be decoded without errors. Frames are read at the selected processing rate. If the custom training set is used, frames are annotated with boxes and class labels and split into training, validation, and test subsets. The same protocol is used for the test videos to ensure the fair comparison.

For the inference, frames are resized or transformed to fit the YOLO input requirements. The detector produces candidate boxes and confidence values. The confidence filtering discards weak detections, while the non-maximum suppression filters out overlapping boxes referring to the same object. Only the needed vehicle classes are passed to the tracker. The threshold and preprocessing settings have to be logged because they influence missed vehicles and false detections.

4.4 YOLO Model and Inference Procedure

YOLO predicts object locations and classes from the input image [1]. In the project, the selected model accepts one traffic frame and produces the vehicle detections required by the pipeline. Every accepted detection has a box, class, and confidence value. Lower threshold retains weak detections, while higher one eliminates them along with some false positives. The chosen threshold has to be logged and used consistently in the comparative experiments.

The detector sees one frame, but counting requires a short history. The accepted boxes are sent to the tracker, which compares them with active tracks in the previous frames. A successful association preserves the vehicle identifier. Even if the other object occludes the vehicle temporarily, the tracker may preserve the track, depending on the tracker and its settings.

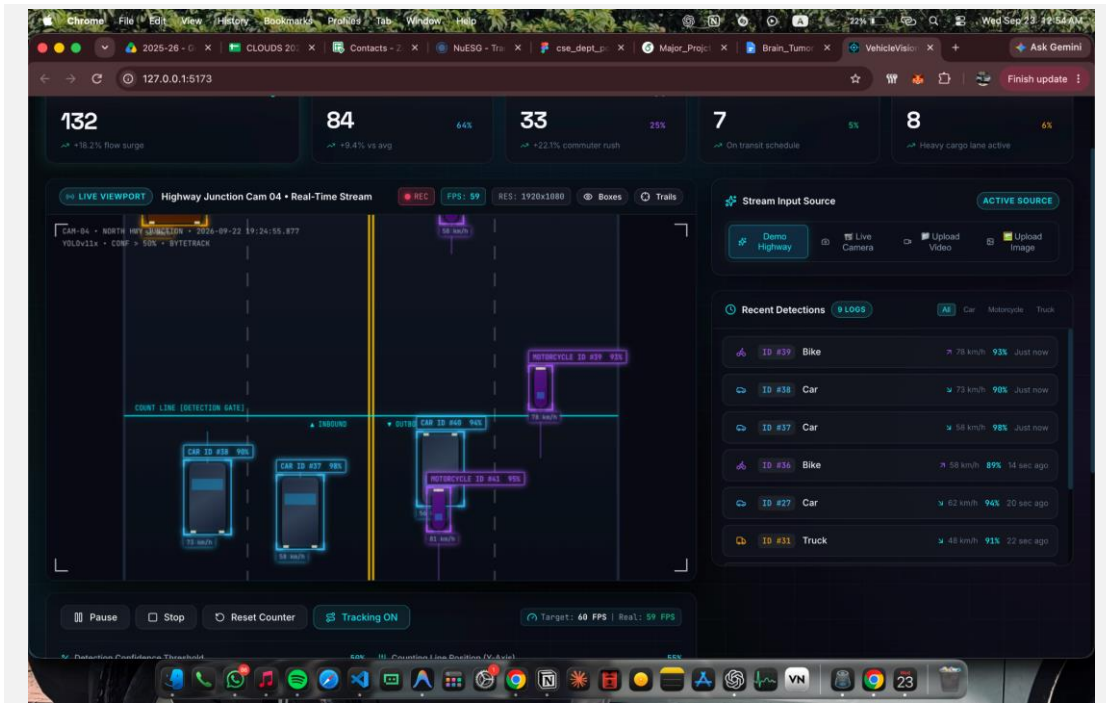


Fig 4.3 Prototype tracking screen with vehicle IDs, confidence values, and the counting line.

Source: Original project report.

4.5 Tracking, Counting, and Direction Analysis

The tracker transforms frame detections into trajectories. Each active vehicle gets an identifier, and the center of its bounding box provides a simple position to check the movement. A line is drawn across the road in the point where vehicles are expected to pass. As the new frames come, the previous and current centers are compared.

If the trajectory crosses the line, the identifier is included in the set of counted vehicles and corresponding counter is incremented. The detection of the same vehicle later does not trigger another event in this direction. The record may also distinguish vehicles traveling in the opposite directions and provide totals for cars, motorcycles, buses, and trucks.

Event Condition	System Action
Vehicle detection passes the confidence filter	Send the box, class, and confidence to tracking
Detection is associated with an active track	Update the identifier and trajectory
Track crosses the configured line	Record one counting event
Identifier has already been counted	Do not increment the same event again
Movement crosses in the opposite direction	Update the opposite directional count

Table 4: Counting-event logic

4.6 Quantitative Metrics

Detection, tracking, and counting require different measurements. Precision, recall, F1-score, mean Average Precision, and Intersection over Union describe detection. Precision answers what percentage of reported boxes are correct, whereas recall answers what percentage of reference vehicles were detected. IoU describes the overlap between the predicted box and its reference. None of these measurements, taken alone, tell us if the same vehicle was counted twice.

Counting must be evaluated using a manually verified total for each test video. Counting error is the difference between predicted and reference counts. If an application-specific value for accuracy is

provided, the formula should be specified along with the result. Frames per second must be recorded separately, as an accurate detector may still be too slow for real-time counting.

Metric	Purpose
Precision	Measures correctness of positive vehicle detections
Recall	Measures vehicles successfully detected
IoU	Measures overlap of predicted and reference boxes
mAP	Summarises object-detection performance
Counting error	Difference between predicted and reference counts
Counting accuracy	Measures agreement with the ground-truth count
FPS	Measures video-processing speed

Table 5: Evaluation measures for the proposed pipeline

The source report describes the measures and recommends using manually verified traffic videos in testing. It does not give a numerical benchmark for the completed prototype. This limitation is retained in this chapter. Without a video, ground truth, model settings, and processing device, a result cannot be reproduced.

4.7 System Implementation

A Python implementation connects the PyTorch-based YOLO interface to OpenCV for video reading, processing, and annotation. ByteTrack or BoT-SORT is used for maintaining identities, whereas NumPy is used for coordinate calculations. OpenCV or Matplotlib can be used for displaying the processed output. Library revisions and model settings should be recorded in experiment record.

Sequentially read frames are passed to the detector. After filtering, the tracker associates current observations with existing tracks. The center of each tracked box is compared with the configured line. A successful crossing is counted as an event and recorded. The processed frame is annotated with the box, class, identifier, line, and running statistics.

5. Results and Analysis

5.1 Detection and Counting Interface

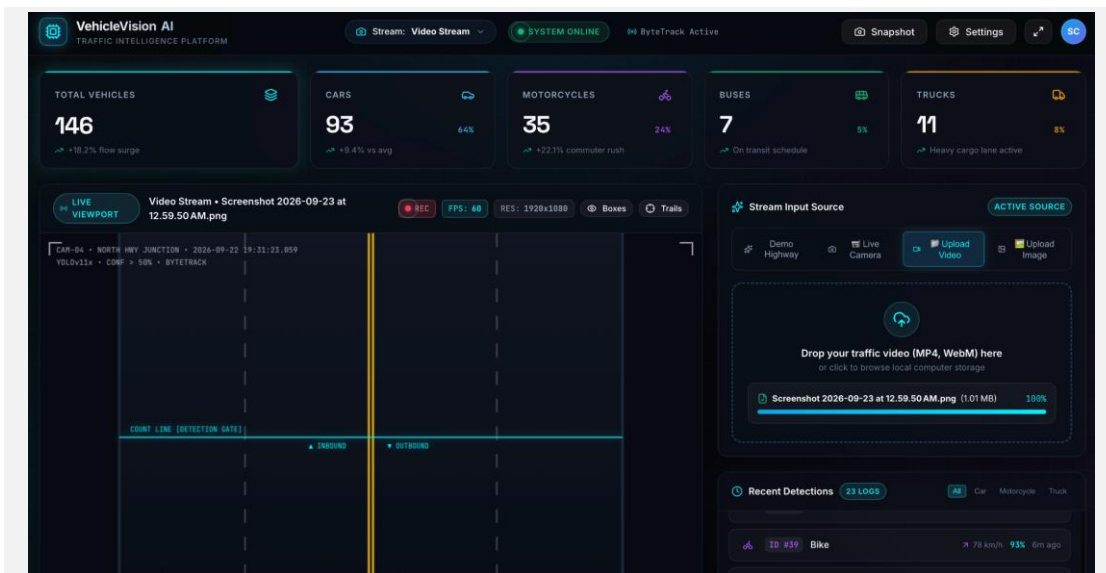


Fig 5.1 Prototype dashboard showing annotated video and class-wise vehicle counts.

Source: Original project report.

The prototype view places the traffic video next to the information provided by the pipeline. Boxes mark detected vehicles, labels identify their classes, and confidence values are the output of the detector. Identifiers let the user track one vehicle across multiple frames. The statistics panel can display total counts and, if implemented, class or direction values.

The number becomes more credible when accompanied by visual evidence. A total number cannot tell if the right vehicles were detected and if the track crossed the line at the right time. The annotated frame lets the user verify both and explain any count changes.

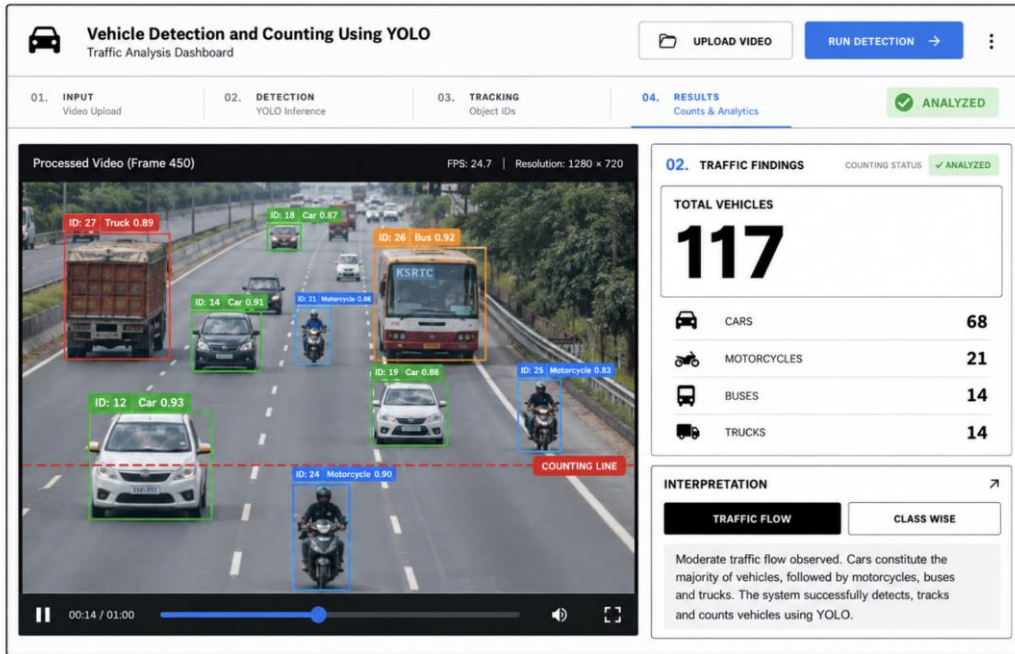


Figure 7. Findings panel showing detected vehicles with tracking IDs, counting line and class-wise counts.
Source: Authors' experiments.

Fig 5.2 Findings panel showing tracked vehicles, counting line, and class-wise counts.
Source: Original project report.

5.3 Findings, Data, and Pipeline Views

The output consists of two parts. The video view contains the original scene, vehicle boxes, class labels, tracking IDs, and the counting boundary. The numerical view contains the total and, if implemented, class-wise and directional values. Separation of the views helps to distinguish missing boxes from identity changes or counting errors.

View	Information Shown	Reason for Separation
Detection	Boxes, classes, and confidence	Verify localization and classification
Tracking	Vehicle IDs and trajectories	Verify identity association
Counting	Virtual line and event totals	Verify counting events
Summary	Total, class-wise, and directional counts	Interpret traffic movement

Table 6: Information presented by the prototype views

5.4 Experimental Evaluation

A complete evaluation requires traffic videos with manually verified counts. The model revision, confidence threshold, tracker settings, and the counting line should stay the same unless they are being tested. The video set should have clear scenes and difficult cases, such as overlapping vehicles, shadows, low lighting, motion blur, and distant objects.

The source report specifies the above procedure but does not give the final precision, recall, counting error, or FPS. A defensible conclusion is that the project produces a prototype which can be evaluated and which exposes the intermediate evidence necessary for checking. Values visible in the interface screenshots are demonstration outputs, not an accuracy benchmark. The results table can be completed after the ground truth preparation.

Evaluation Item	Required Record	Status in Source Report
Detection Quality	Precision, recall, F1-score, mAP, and IoU	Metric protocol defined; no numeric result supplied
Counting Quality	Reference count, predicted count, and error	Ground-truth procedure defined; no numeric result supplied
Processing Speed	Frames per second and execution setup	Recommended measurement; no formal benchmark supplied
Robustness	Lighting, occlusion, density, and viewpoint cases	Factors identified for future testing

Table 7: Evaluation status based on the project report

6. Discussion

The project combines a frame-level detector, a tracker, and a line-crossing rule into one traffic counting workflow. YOLO provides the boxes and class labels. Tracking provides a short-term identity, and the virtual line converts movement into an event. The user can trace the path which leads to the count instead of seeing just a final total.

This visibility also shows the weaknesses of the design. Overlapping vehicles can confuse the detector or cause an identity switch. Low lighting and small distant vehicles reduce box quality. Camera movements change the scene geometry, and a line suitable for one road may not be suitable for another. These conditions should be measured during testing.

The prototype should be considered as a configured traffic analysis workflow, not as a universal solution. Detection quality and counting quality are separate aspects. The detector may detect the vehicle while the tracker cannot maintain its identity, and a stable track can still create a wrong event due to misconfigured line. Separate metrics and views show this difference.

7. Applications

The prototype can provide preliminary traffic-flow monitoring from the recorded video or a suitable camera stream. It can produce total counts, class-wise summaries, and directional information for the selected road region. Such outputs are appropriate for academic demonstrations and for early studies of congestion, road surveillance, and intelligent transportation systems.

The same pipeline can be used as the baseline for comparing models or trackers in identical video conditions. One component can be changed while the counting and display logic stay the same. Count histories, lane-specific summaries, and report export are potential extensions. But every new output should be evaluated against the appropriate ground truth.

8. Conclusion

This chapter has described a YOLO-based workflow for vehicle detection and counting. Traffic video is read frame-by-frame, vehicles are classified, detections are tracked over time, and the count is registered when the trajectory crosses the selected boundary. Boxes, identifiers, and events stay visible for inspection. The same records can be used for class-wise and directional summaries.

The source project report provides the architecture, requirements, implementation details, and evaluation plan. It does not provide a completed numerical benchmark, thus this chapter does not

claim an accuracy or speed value. At the moment, the result is an inspectable prototype for educational and research purposes.

9. Future Scope

The input can be later extended to CCTV or IP-camera streams. Other extensions may include lanewise counting, speed estimation, and traffic density measurement. Additional event rules can be defined for wrong-way movement, stopped vehicles, traffic violations, or accident-like situations. Storing counts can be useful for identifying the busy periods and studying congestion.

Before deploying the pipeline in practice, the detector and tracker should be tested under different lighting conditions, viewpoints, vehicle sizes, occlusion levels, and traffic densities. Measurements of FPS, inference latency, memory consumption, and CPU or GPU usage will determine the deployment cost. Broader data sets, improved identity association, adaptive counting regions, and explanation views will provide a better ground for field tests.

10. References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [2] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.
- [3] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.
- [4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020.
- [5] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023.
- [6] C.-Y. Wang et al., "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," arXiv preprint arXiv:2402.13616, 2024.
- [7] Y. Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," European Conference on Computer Vision (ECCV), 2022.
- [8] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," IEEE International Conference on Image Processing (ICIP), 2017.
- [9] A. Bewley et al., "Simple Online and Realtime Tracking," IEEE International Conference on Image Processing, 2016.
- [10] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," European Conference on Computer Vision (ECCV), 2014.